

Task 3

Static Timing Analysis (STA) & Timing Closure

8-bit Arithmetic Logic Unit (alu_8bit) — Gate-Level Timing Verification

Toolchain: Yosys 0.52 • OpenSTA 2.0.17 • SkyWater Sky130 (sky130_fd_sc_hd, tt_025C_1v80) • WSL2 / Ubuntu

1. Goal

Run static timing analysis (STA) on the synthesized gate-level netlist of the 8-bit ALU using a real standard-cell timing library, identify the worst negative slack (WNS) and any timing violations, trace the critical path, and propose concrete fixes.

2. Design Note — Combinational Circuit & Virtual Clock

alu_8bit is a purely combinational design (no internal flip-flops or registers). Since STA setup/hold analysis is fundamentally clock-relative, a virtual clock was defined in the SDC constraints purely as a timing reference — it does not drive any physical pin in the design. This is standard practice for analyzing the propagation delay of a combinational block: the virtual clock period sets a time budget, and STA reports whether the longest input-to-output path fits inside that budget.

3. Re-Synthesis with a Real Timing Library

Task 2's netlist used generic logic gates (AND/OR/XOR/NOT) with no timing information. For STA, the design was re-synthesized using Yosys mapped to the open-source SkyWater Sky130 high-density standard cell library (sky130_fd_sc_hd__tt_025C_1v80.lib), which provides real silicon-characterized gate delays.

Synthesis Script — synth_timing.ys

```
read_liberty -lib sky130.lib
read_verilog alu_8bit.v
hierarchy -check -top alu_8bit
proc
opt
techmap
opt
abc -liberty sky130.lib
opt_clean
write_verilog -noattr netlist_timing.v
stat -liberty sky130.lib
```

Result

Yosys imported 418 cell types from the Sky130 library and mapped the ALU to 225 real standard cells (vs. 468 generic gates in Task 2) — fewer cells because real standard cells implement complex Boolean functions (e.g. AND-OR-INVERT, majority gates) in a single cell.

Metric	Value
--------	-------

Total cells	225 (sky130_fd_sc_hd library cells)
Chip area	1311.26 μm^2
Sequential element area	0% (fully combinational design)
Key cell types used	maj3_1, xor2_1, nand2_1, nor2_1, a21oi_1, mux2_1, and more

4. Timing Constraints (SDC)

A baseline SDC file was written defining a 100 MHz virtual clock (10 ns period) with 1 ns of input/output delay margin, representing a reasonable starting target frequency for an academic ALU design.

constraints.sdc

```
create_clock -name virt_clk -period 10
set_input_delay -clock virt_clk 1 [all_inputs]
set_output_delay -clock virt_clk 1 [all_outputs]
```

The same structure was reused at tighter clock periods (5 ns and 4 ns) to determine the design's real maximum operating frequency and to deliberately surface a timing violation for analysis.

5. STA Script & Execution

OpenSTA (the open-source equivalent of Synopsys PrimeTime) was used to load the Sky130 liberty file, the timing-aware netlist, and the SDC constraints, then report setup (max-delay) and hold (min-delay) checks along with WNS/TNS.

sta_run.tcl

```
read_liberty sky130.lib
read_verilog netlist_timing.v
link_design alu_8bit
read_sdc constraints.sdc
report_checks -path_delay max -format full_clock_expanded
report_checks -path_delay min -format full_clock_expanded
report_tns
report_wns
```

Command

```
sta sta_run.tcl
```

6. Timing Results Across Target Frequencies

STA was run at three clock periods to characterize the design's timing margin and locate its real operating limit.

Clock Period	Target Frequency	Critical Path Delay	WNS	TNS	Status
10 ns	100 MHz	3.89 ns	0.00 ns (+5.11 ns slack)	0.00 ns	MET — comfortable margin
5 ns	200 MHz	3.89 ns	0.00 ns (+0.11 ns slack)	0.00 ns	MET — marginal
4 ns	250 MHz	3.89 ns	-0.89 ns	-5.21 ns	VIOLATED

The critical path delay (3.89 ns) is constant across all three runs, as expected — it is a property of the physical netlist, not the clock constraint. What changes is whether the required time (set by the clock period) is large enough to accommodate it.

7. Critical Path Analysis

Worst-case max-delay path, reported by OpenSTA (startpoint b[1] → endpoint zero):

```
Startpoint: b[1] (input port clocked by virt_clk)
Endpoint:   zero (output port clocked by virt_clk)
Path Group: virt_clk | Path Type: max
```

```
Delay  Time  Cell
0.15   1.15  sky130_fd_sc_hd__clkinv_1
0.43   1.57  sky130_fd_sc_hd__maj3_1
0.39   1.96  sky130_fd_sc_hd__maj3_1
0.39   2.35  sky130_fd_sc_hd__maj3_1
0.39   2.74  sky130_fd_sc_hd__maj3_1
0.39   3.13  sky130_fd_sc_hd__maj3_1
0.40   3.53  sky130_fd_sc_hd__maj3_1
0.17   3.70  sky130_fd_sc_hd__xor2_1
0.14   3.84  sky130_fd_sc_hd__o21ai_0
0.05   3.89  sky130_fd_sc_hd__nor3_1
       3.89  data arrival time (TOTAL)
```

The critical path runs through six chained maj3_1 (majority/full-adder) cells. This is the carry-propagation chain inside the ADD/SUB logic: each bit's carry-out depends on the previous bit's carry-in, forming a serial dependency chain that is inherently the slowest structure in a ripple-style adder. The remaining delay comes from the XOR/AND-OR-INVERT logic that derives the zero flag from the final result.

8. Identified Violation

At a 4 ns clock period (250 MHz target), the same 3.89 ns critical path exceeds the 3.00 ns required time (4 ns period minus 1 ns output delay margin):

```
3.00  data required time
-3.89  data arrival time
-----
```

```
-0.89    slack (VIOLATED)
```

```
tns -5.21
wns -0.89
```

WNS = -0.89 ns indicates the design's true maximum safe operating point sits between 200 MHz (passes with 0.11 ns margin) and 250 MHz (fails by 0.89 ns) — roughly 220-230 MHz, well short of the 250 MHz target.

9. Proposed Fixes

1. Pipelining the carry chain — Insert a pipeline register partway through the adder (e.g. after bit 3's carry), splitting the 6-stage maj3_1 chain into two 3-stage halves across two clock cycles. This roughly halves the critical path delay at the cost of one additional cycle of latency and a small area overhead for the pipeline register.
2. Carry-lookahead or carry-select adder — Replace the ripple-carry structure (implied by the chained maj3_1 cells) with a carry-lookahead adder (CLA) or carry-select adder. These compute carries in parallel rather than propagating them serially bit-by-bit, cutting the dependency chain from $O(n)$ to roughly $O(\log n)$ gate delays for an 8-bit operand.
3. Timing-driven re-synthesis — Re-run Yosys/ABC with a tighter delay target (abc -liberty sky130.lib -D <target_ps>) so the optimizer is explicitly pushed to select higher drive-strength cell variants along the critical path, trading area/power for speed.
4. Logic restructuring of the zero-flag path — The final XOR → AND-OR-INVERT → NOR chain that derives the zero flag adds ~0.36 ns on top of the adder delay. Computing zero directly from the carry-chain's intermediate signals (rather than waiting for the full registered result) could shave a small additional margin.

10. Deliverables Summary

Deliverable	Result
1. Timing report with WNS (if any)	WNS = -0.89 ns at 250 MHz (4 ns period); 0.00 ns (margin) at 100/200 MHz
2. Write-up describing fixes	Pipelining, carry-lookahead adder, timing-driven re-synthesis, zero-flag restructuring (Section 9)

11. Conclusion

STA on the Sky130-mapped netlist shows the 8-bit ALU comfortably meets timing at 100 MHz (5.11 ns slack) and marginally meets timing at 200 MHz (0.11 ns slack), with a critical path of 3.89 ns running through the ripple-carry chain of the ADD/SUB logic. Pushing to 250 MHz produces a clear, reproducible violation (WNS = -0.89 ns), confirming the design's real performance ceiling sits just above 200 MHz. The dominant bottleneck — six chained maj3_1

carry cells — is a textbook ripple-carry-adder limitation, and is addressable through pipelining or a carry-lookahead/carry-select adder structure without changing the ALU's functional behavior.